

Problems with Multi-Threading

↳ ① Race Condition

↳ Program $\rightarrow$ Final state.

Depend on the order of
exⁿ of threads.

```
int Count = 0;
```

```
Count ++;
```

→ Read Count from Memory.

→ Increment Count by 1.

→ Write updated value back

```
class Counter {
```

```
    int Count = 0;
```

```
    void increment() {
```

```
        Count ++;
```

```
    }
```

```
Counter c1 = new Counter();
```

```
c1.increment();
```

```
class Counter {
```

```
    int Count = 0;
```

```
    void increment() {
```

```
        Count ++;
```

```
    }
```

```
Counter c1 = new Counter();
```

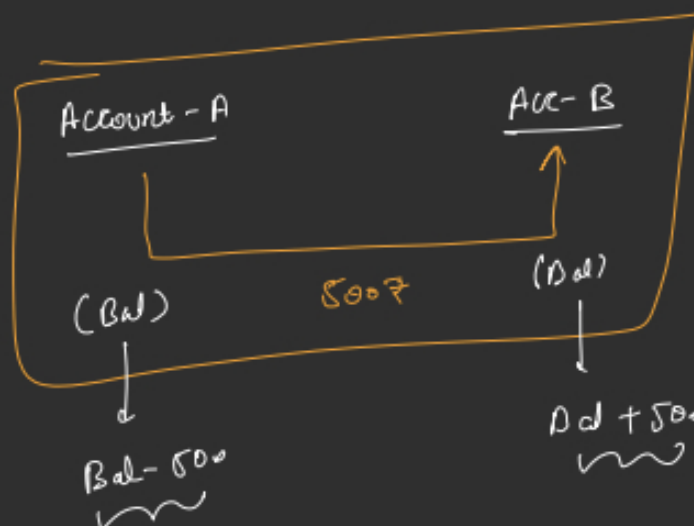
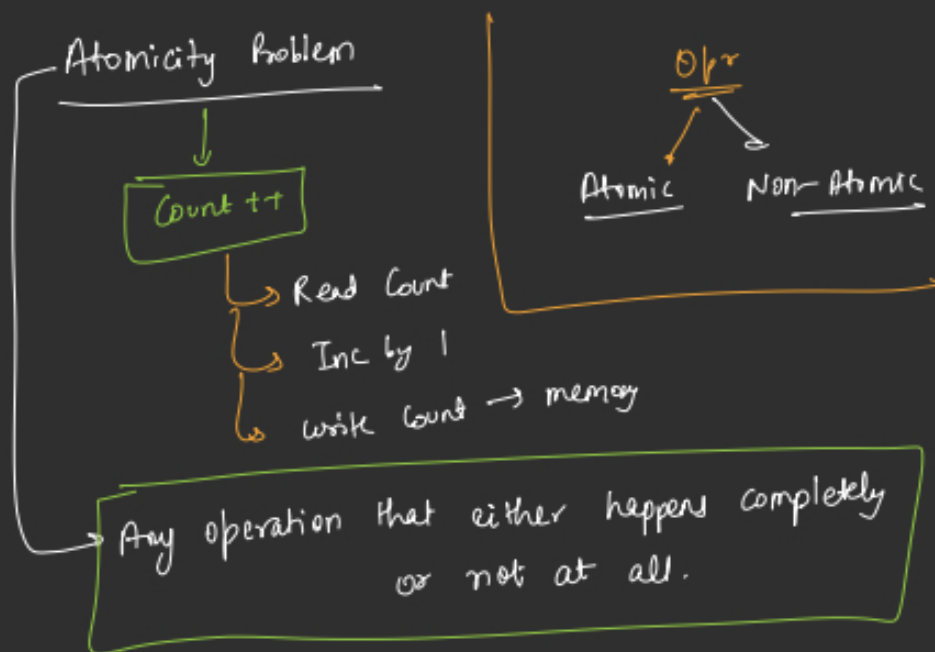
```
c1.increment();
```

Steps	T1	T2
①	Read C=0	
②		Read C=0
③	C=C+1	
④		C=C+1
⑤	Write ①	
⑥		Write ②

```
void increment() {
    count++;
}
```

Shared Resource

List<Integer> list
BankAccount account;



→ Synchronized
→ Atomic Integer

Atomic

↳ assignment

(int x = 2)

↳ Student s1 = new Student();

T1 [500]

T2 [600]

bal ~~1000~~ 400
-100

Non-atomic

↳ $x++$, $y--$

↳ $x = x + 5$

check-then-act

if (bal >= amt) {
 bal - amt;
}

double d = 112.234;

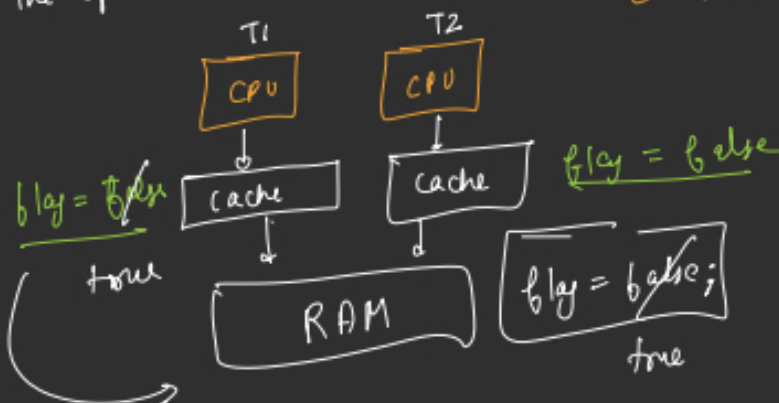
↳ JVM
64-bit

32-bit | 32-bit

→ Visibility

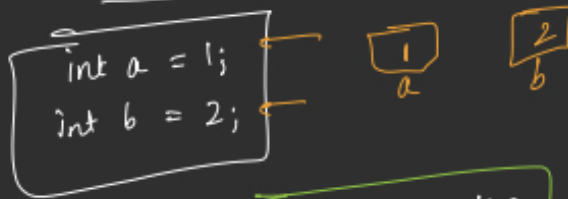
↳ One thread updates a variable, but another thread does not see the updated value.

volatile
synchronized



→ Ordering

↳ Sequence in which instructions are executed.



JVM →

